DEPARTMENT OF COMPUTER SCIENCE

KESHAV MAHAVIDYALAYA

(UNIVERSITY OF DELHI)

BLOCKCHAIN

"Only Knowledge can provide salvation"

Submitted by

Satyam yadav

Roll No.: 22035570075

B.Sc. (Hons) Computer Science

P1 : Write a program to demonstrate sending a digitally signed document, including signing and verification.

Code:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding from
cryptography.hazmat.primitives import hashes
import hashlib
import base64

print("\nGenerating RSA Key Pair")
private_key = rsa.generate_private_key(
public_exponent=65537,
key_size=2048
)
public_key = private_key.public_key()
print("PUBLIC KEY GENERATED\n")


print("Enter message to sign:")
user_input = input(">> ")

if user_input.strip() == "":
user_input = "Humpty dumpty sat on the wall"

document = user_input.encode()
print("Document:", user_input, "\n")


print("Hashing Document (SHA-256)...")
doc_hash = hashlib.sha256(document).hexdigest()
print("Hash:", doc_hash, "\n")


print("Signing Document...")
signature = private_key.sign(
document,
padding.PSS(
mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH
),
hashes.SHA256()
)

signature_b64 = base64.b64encode(signature).decode()
print("Signature :", signature_b64[:60], "...\n")

try:
public_key.verify(
signature,
```

```
document,
padding.PSS(
mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH ),
hashes.SHA256()
)
print("Signature VALID\n")

except Exception:
print("Signature INVALID\n")


print("Do you want to test tampering?
(y/n)") choice = input(">> ").lower()

if choice == "y":
tampered_input = input("Enter tampered message: ")
tampered_doc = tampered_input.encode()

try:
public_key.verify(
signature,
tampered_doc,
padding.PSS(
mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH ),
hashes.SHA256()
)
print("Signature VALID (Unexpected)\n")

except Exception:
print("Signature INVALID (Tampering Detected)\n")
```

```
Generating RSA Key Pair
PUBLIC KEY GENERATED

Enter message to sign:
>> mister mouse
Document: mister mouse

Hashing Document (SHA-256)...
Hash: d136e29647a9c213cc13676099926bcbeb8d265e95c12a19753ecfcde9fa5cae

Signing Document...
Signature : KaE9iHTWPjARkjCHUOFV63U/ABMt7lr++L54d5Ehm0OgmpXSq9AxQ/3qpzvQ ...

Signature VALID

Do you want to test tampering? (y/n)
>> y
Enter tampered message: mister mouse
Signature VALID (Unexpected)
```

P2: Create a blockchain containing block hash, transaction history, time of creation

```
import hashlib
import datetime
import json

class Block:
    def __init__(self, index, transactions, timestamp,
        previous_hash): self.index = index
        self.transactions = transactions
        self.timestamp = timestamp
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "transactions": self.transactions,
            "timestamp": str(self.timestamp),
            "previous_hash": self.previous_hash
        }, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, ["Genesis Block"], datetime.datetime.now(), "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, transactions):
        latest_block = self.get_latest_block()
        new_block = Block(
            latest_block.index + 1,
            transactions,
            datetime.datetime.now(),
            latest_block.hash
        )
        self.chain.append(new_block)
    def print_chain(self):
        for block in self.chain:
            print(block.index)
            print(block.timestamp)
```

```
        print(block.transactions)
        print(block.previous_hash)
        print(block.hash)
        print()

bc = Blockchain()
bc.add_block(["Alice pays Bob ₹500", "Bob pays Charlie ₹200"])
bc.add_block(["Charlie pays Dave ₹100"])
bc.print_chain()
```

Output:

```
0
2026-04-06 19:40:01.541326
['Genesis Block']
0
84864cc2c4a85d6fd078e3a652d7fc384748642b36f32b05b23aa769f969617a

1
2026-04-06 19:40:01.541326
['Alice pays Bob ₹500', 'Bob pays Charlie ₹200']
84864cc2c4a85d6fd078e3a652d7fc384748642b36f32b05b23aa769f969617a
f7f0cd492fa884f6b2e6092ff2c684027ab8e649e4f4a7ac36a11eec1e8d0a58

2
2026-04-06 19:40:01.541326
['Charlie pays Dave ₹100']
f7f0cd492fa884f6b2e6092ff2c684027ab8e649e4f4a7ac36a11eec1e8d0a58
cfb139f070f006c2dc71263dda92a4b9b5d97ea088bc3ec5bbaae27b24aa2d0f
```

LAB Exercise 3

Q1. Write a C++ program to generate SHA-256 hash using any available library.

Q2. Using SHA 256, obtain the message digest of string “Blockchain Developer”.

Q3. Generate SHA-256 hashes for:

- "Hello World"

- "Hello world"

```
import hashlib

def generate_sha256(text):
```

```
encoded_text = text.encode('utf-8')


sha256_hash = hashlib.sha256(encoded_text)


return sha256_hash.hexdigest()


message = input("Enter a message: ")
print("SHA-256 Hash:", generate_sha256(message))


import hashlib

text = "Blockchain Developer"
hash_value =

                    hashlib.sha256(text.encode()).hexdigest

                    () print("SHA-256 Hash:", hash_value)


import hashlib

text1 = "Hello World"
text2 = "Hello world"

hash1 =
                    hashlib.sha256(text1.encode()).hex
                    digest() hash2 =
                    hashlib.sha256(text2.encode()).hex
                    digest()

print("Hello World ->", hash1)
print("Hello world ->", hash2)
```

```python
import hashlib

text = "Blockchain Developer"
hash_value = hashlib.sha256(text.encode()).hexdigest()

print("SHA-256 Hash:", hash_value)
```

```
SHA-256 Hash: 0e3f62dab611686a781b2b9fd2a29544ed9829147b104cc4ad6315d88cde37e0
```

```python
import hashlib

text1 = "Hello World"
text2 = "Hello world"

hash1 = hashlib.sha256(text1.encode()).hexdigest()
hash2 = hashlib.sha256(text2.encode()).hexdigest()

print("Hello World  ->", hash1)
print("Hello world  ->", hash2)
```

```
Hello World  -> a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Hello world  -> 64ec88ca00b268e5ba1a35678a1b5316d212f4f366b2477232534a8aeca37f3c
```

```python
import hashlib

def generate_sha256(text):
    # Encode the input string to bytes
    encoded_text = text.encode('utf-8')

    # Create SHA-256 hash object
    sha256_hash = hashlib.sha256(encoded_text)

    # Get hexadecimal digest
    return sha256_hash.hexdigest()

# Example usage
message = input("Enter a message: ")
print("SHA-256 Hash:", generate_sha256(message))
```

```
Enter a message: satyam
SHA-256 Hash: 8ecdbae10effd3c0cd8453a86a3c90b73cf365e6e4fc796ee814eac71dd4e539
```

LAB Exercise 4

Q1.Write a program to generate SHA-256 hash of “Hello World”

import hashlib

Input string

text = "Hello World"

Generate SHA-256 hash

hash_object = hashlib.sha256(text.encode())

Get hexadecimal digest

hash_hex = hash_object.hexdigest()

# Print result
print("Original Text:", text)
print("SHA-256 Hash:", hash_hex)

```python
import hashlib

# Input string
text = "Hello World"

# Generate SHA-256 hash
hash_object = hashlib.sha256(text.encode())

# Get hexadecimal digest
hash_hex = hash_object.hexdigest()

# Print result
print("Original Text:", text)
print("SHA-256 Hash:", hash_hex)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
Original Text: Hello World
SHA-256 Hash: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Run completed in 4104.600000000093ms
```

LAB Exercise 5

Use any programming language to implement the Question:

Q1. Write a program to implement RSA encryption and decryption where:

- p = 17, q = 11
- Choose e = 7

Perform the following:

Calculate: n and φ(n)

Find the

Encrypt the message: M = 8

Decrypt the ciphertext and verify that you get the original message.
EVERY STUDENT TAKE DIFFERENT VALUE OF "M" AND "e"

NAME : SATYAM YADAV
COURSE: BSC(HONS) COMPUTER SCIENCE

RSA Encryption and Decryption

1. Aim
To implement RSA algorithm.

2. Given Values
p = 17, q = 11, e = 7, M = 8

3. Calculations
n = 187

$\varphi(n) = 160$
$d = 23$

4. Encryption
$C = 134$

5. Decryption
$M = 8$

6. Code
```
# RSA Implementation

p = 17
q = 11
e = 7
M = 8

# Step 1
n = p * q
phi = (p - 1) * (q - 1)

# Step 2: Find d
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d

d = mod_inverse(e, phi)

# Step 3: Encryption
C = pow(M, e, n)

# Step 4: Decryption
M_decrypted = pow(C, d, n)

# Output
print("p =", p, "q =", q)
print("n =", n)
print("phi(n) =", phi)
print("e =", e)
print("d =", d)
print("Original Message =", M)
print("Encrypted Message =", C)
print("Decrypted Message =", M_decrypted)
```

7. Output

```
p = 17 q = 11
n = 187
phi(n) = 160
e = 7
d = 23
Original Message = 8
Encrypted Message = 134
Decrypted Message = 8
```

LAB Exercise 6

Q1. Create a simple blockchain using Proof of Work (PoW) and perform the following:
- Create with transaction data (e.g., A → B, B → C, C → D).
- Set (hash must start with "00").
For each block:

- Calculate hash
- Perform mining (PoW)
- Link with previous block

```
import hashlib

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block()

    def calculate_hash(self):
        value = str(self.index) + self.data + self.previous_hash + str(self.nonce)
        return hashlib.sha256(value.encode()).hexdigest()
```

```
    def mine_block(self):
        print(f"\nMining Block {self.index}...")
        while True:
            hash_value = self.calculate_hash()
            if hash_value.startswith("00"):
                print(f"Block {self.index} mined with nonce {self.nonce}")
                return hash_value
            else:
                self.nonce += 1


class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = Block(0, "Genesis Block", "0")
        self.chain.append(genesis_block)

    def add_block(self, data):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)

    def print_chain(self):
        print("\n===== BLOCKCHAIN =====")
        for block in self.chain:
            print("\nIndex:", block.index)
            print("Data:", block.data)
            print("Hash:", block.hash)
            print("Previous Hash:", block.previous_hash)
            print("Nonce:", block.nonce)

blockchain = Blockchain()


blockchain.add_block("A -> B")
blockchain.add_block("B -> C")
blockchain.add_block("C -> D")


blockchain.print_chain()
```

```python
import hashlib

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block()

    def calculate_hash(self):
        value = str(self.index) + self.data + self.previous_hash + str(self.nonce)
        return hashlib.sha256(value.encode()).hexdigest()

    def mine_block(self):
        print(f"\nMining Block {self.index}...")
        while True:
            hash_value = self.calculate_hash()
            if hash_value.startswith("00"):
                print(f"Block {self.index} mined with nonce {self.nonce}")
                return hash_value
            else:
                self.nonce += 1


class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = Block(0, "Genesis Block", "0")
        self.chain.append(genesis_block)

    def add_block(self, data):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)

    def print_chain(self):
        print("\n===== BLOCKCHAIN =====")
        for block in self.chain:
            print("\nIndex:", block.index)
            print("Data:", block.data)
            print("Hash:", block.hash)
            print("Previous Hash:", block.previous_hash)
            print("Nonce:", block.nonce)


# Create blockchain
blockchain = Blockchain()

# Add transactions
blockchain.add_block("A -> B")
blockchain.add_block("B -> C")
blockchain.add_block("C -> D")
```

Console

```
Mining Block 0...
Block 0 mined with nonce 642

Mining Block 1...
Block 1 mined with nonce 68

Mining Block 2...
Block 2 mined with nonce 291

Mining Block 3...
Block 3 mined with nonce 403

===== BLOCKCHAIN =====

Index:
 0
Data: Genesis Block
Hash: 00fdcb7606e2fd2a721926e16edbfca1cd83b534e9352aa144eb28ac1819fa54
Previous Hash: 0
Nonce: 642

Index:
 1
Data: A -> B
Hash: 00f2a803049746798e144fe4230326e517b7483436c4ef13a4354c14f54539b1
Previous Hash: 00fdcb7606e2fd2a721926e16edbfca1cd83b534e9352aa144eb28ac1819fa54
Nonce: 68

Index:
 2
Data: B -> C
Hash: 0070c3fd4eac420c36bc6559654bdfb27ec13dadad26b41e8402b900bc9384da
Previous Hash: 00f2a803049746798e144fe4230326e517b7483436c4ef13a4354c14f54539b1
Nonce: 291

Index:
 3
Data: C -> D
Hash: 00108190c02e08c6c16bfde55d6cd98d36f261aeff9c0e1dac57c311daa8d8bd
Previous Hash: 0070c3fd4eac420c36bc6559654bdfb27ec13dadad26b41e8402b900bc9384da
Nonce: 403
```

LAB Exercise 7

Use any programming language to implement the Question:

Q1. Create a blockchain of 5 blocks, print hash of each block, and verify chain integrity.
Q2.

Tamper with 3rd block and show how validation fails.

Q2. Create a blockchain having 5 nodes and check its validity.

Question 1

```
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        value = str(self.index) + str(self.timestamp) + self.data + self.previous_hash
        return hashlib.sha256(value.encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def add_block(self, data):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)

    def is_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            # Check current hash
            if current.hash != current.calculate_hash():
                return False

            # Check chain link
```

```
        if current.previous_hash != previous.hash:
            return False

    return True

blockchain = Blockchain()

for i in range(1, 5):
    blockchain.add_block(f"Block {i} Data")

print("Blockchain Hashes")
for block in blockchain.chain:
    print(f"Block {block.index} Hash: {block.hash}")

print("\nIs Blockchain Valid?", blockchain.is_valid())

print("\nTampering with Block 3...")
blockchain.chain[3].data = "Hacked Data"
blockchain.chain[3].hash = blockchain.chain[3].calculate_hash()

print("Is Blockchain Valid After Tampering?", blockchain.is_valid())
```

```
=== Blockchain Hashes ===
Block 0 Hash: fc5d08ce44b81c39a31a8e021ab14b2c9890aa5f2e23a3735c169709cad98dd1
Block 1 Hash: 514bf074671769783dcae31be4c77b9a26bd52b1521838e39e8470a7338132fd
Block 2 Hash: 1b3033b7653d2cc9109db31cb33752938652fec3274145c2f45031a2e7ce3217
Block 3 Hash: 5f2925a10ece68dd424b9a48f9ba2722383a646a28eab1d7e67773fea8c33aa1
Block 4 Hash: cccecdbc534ef2f7005334cca4354b4335494cad45eebe0423267d7defa4658c

Is Blockchain Valid?
 True

Tampering with Block 3...
Is Blockchain Valid After Tampering? False
```

Question 2 :

```
class Node:
    def __init__(self, name, blockchain):
        self.name = name
        self.blockchain = blockchain

    def validate_chain(self):
        return self.blockchain.is_valid()

# Create 5 nodes
```

```
nodes = []
for i in range(5):
    nodes.append(Node(f"Node-{i+1}", blockchain))

# Check validity on all nodes
print("\n=== Node Validation ===")
for node in nodes:
    print(f"{node.name} Validates Chain:", node.validate_chain())
```

```
Node-1 Validates Chain: False
Node-2 Validates Chain: False
Node-3 Validates Chain: False
Node-4 Validates Chain: False
Node-5 Validates Chain: False
```

LAB Exercise 8

Q1. Implement a smart contract using **solidity programming language.**
Q2. Create a contract to store student name and age. (Use: - string and uint)
Q3. Write a contract to transfer funds between users.

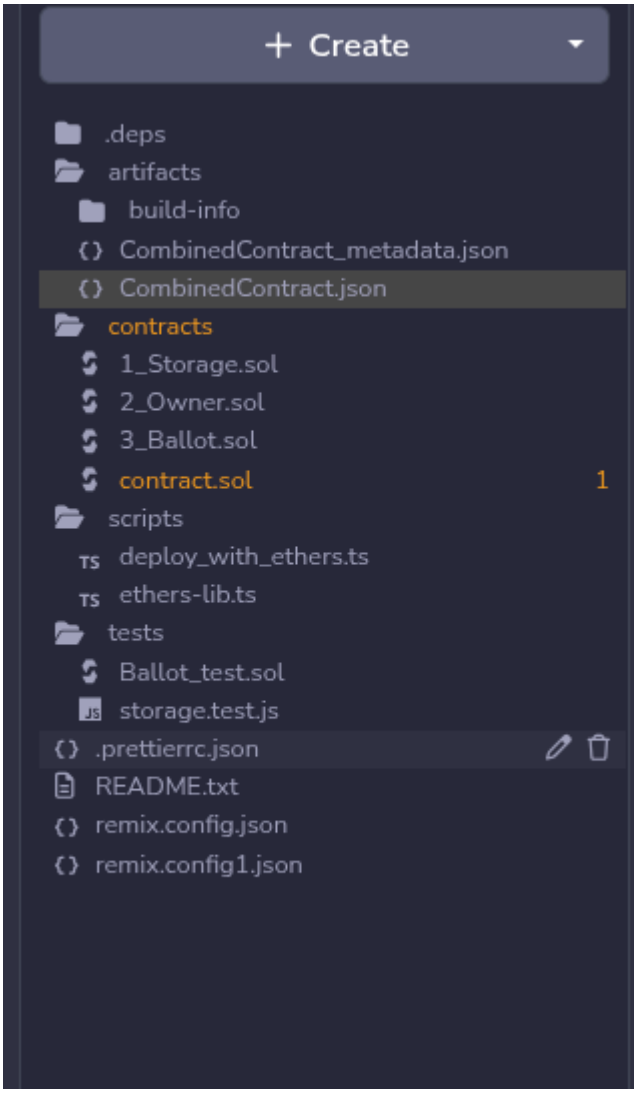
Full contract code :
Contract address: 0xd9145CCE52D386f254917e481eB44e9943F39138
Account used here:
1:0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
2:0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

Code 👍

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CombinedContract {

    // ------------------------------
    // Q1: Simple Storage
    // ------------------------------
    uint public storedData;

    function set(uint x) public {
        storedData = x;
    }

    function get() public view returns (uint) {
        return storedData;
    }

    // ------------------------------
```

```
    // Q2: Student Info
    // -------------------------------
    struct Student {
        string name;
        uint age;
    }

    Student public student;

    function setStudent(string memory _name, uint _age) public {
        student = Student(_name, _age);
    }

    function getStudent() public view returns (string memory, uint) {
        return (student.name, student.age);
    }

    // -------------------------------
    // Q3: Transfer Funds
    // -------------------------------

    // Deposit Ether
    function deposit() public payable {}

    // Check contract balance
    function getBalance() public view returns (uint) {
        return address(this).balance;
    }

    // Transfer Ether
    function transferFunds(address payable _to, uint _amount) public {
        require(address(this).balance >= _amount, "Insufficient
balance");
        _to.transfer(_amount);
    }
}
```

Folder structure:


+ Create
.deps
artifacts
build-info
CombinedContract_metadata.json
CombinedContract.json
contracts
1_Storage.sol
2_Owner.sol
3_Ballot.sol
contract.sol
1
scripts
deploy_with_ethers.ts
ethers-lib.ts
tests
Ballot_test.sol
storage.test.js
.prettierrc.json
README.txt
remix.config.json
remix.config1.json

Q1:

https://remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.1.0 | default_workspace | Connect to GitHub | Sign In BETA | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment | Fork | Reset

Remix VM | Osaka

Account 1 | 0x5B3...eddC4 | 99.999 ETH

Functions

Select a function to interact with...

Low level interaction

get

uint256: 10

Call

Balance 0 ETH

Transactions recorder 4 | Save

Contract call | Transaction list | Newest

CombinedContract | 0xd91...39138 | 4 transactions | Deployed 6min ago

constructor | tx: 0x408...7d31c | deploy | 6min ago

Compile | contract.sol 1 | CombinedContract.json

```
contract CombinedContract {    PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left

    // -------------------------------
    // Q1: Simple Storage
    // -------------------------------
    uint public storedData;

    function set(uint x) public {    22559 gas
        storedData = x;
    }

    function get() public view returns (uint) {    2431 gas
        return storedData;
    }

    // -------------------------------
    // Q2: Student Info
    // -------------------------------
    struct Student {
        string name;
        uint age;
    }

    Student public student;

    function setStudent(string memory _name, uint _age) public {    infinite gas
        student = Student( name,  age);
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

```
revert
    The transaction has been reverted to the initial state.
Note: The called function should be payable if you send value and the value you send should be less than your current balance.
If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...0000a logs: 0 hash: 0x87f...dbd77 | Debug

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c | Debug

Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

https://remix.ethereum.org/#lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.1.0 | default_workspace | Connect to GitHub | Sign In BETA | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment | Fork | Reset

Remix VM | Osaka

Account 1 | 0x5B3...eddC4 | 99.999 ETH

Functions

Select a function to interact with...

Low level interaction

set uint256

10

Value 0 wei

Gas limit auto 0

Transact

Balance 0 ETH

Transactions recorder 5 | Save

Contract call | Transaction list | Newest

CombinedContract | 0xd91...39138 | 5 transactions | Deployed 0min ago

Compile | contract.sol 1 | CombinedContract.json

```
contract CombinedContract {    PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left

    // -------------------------------
    // Q1: Simple Storage
    // -------------------------------
    uint public storedData;

    function set(uint x) public {    22559 gas
        storedData = x;
    }

    function get() public view returns (uint) {    2431 gas
        return storedData;
    }

    // -------------------------------
    // Q2: Student Info
    // -------------------------------
    struct Student {
        string name;
        uint age;
    }

    Student public student;

    function setStudent(string memory _name, uint _age) public {    infinite gas
        student = Student( name,  age);
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

```
Note: The called function should be payable if you send value and the value you send should be less than your current balance.
If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

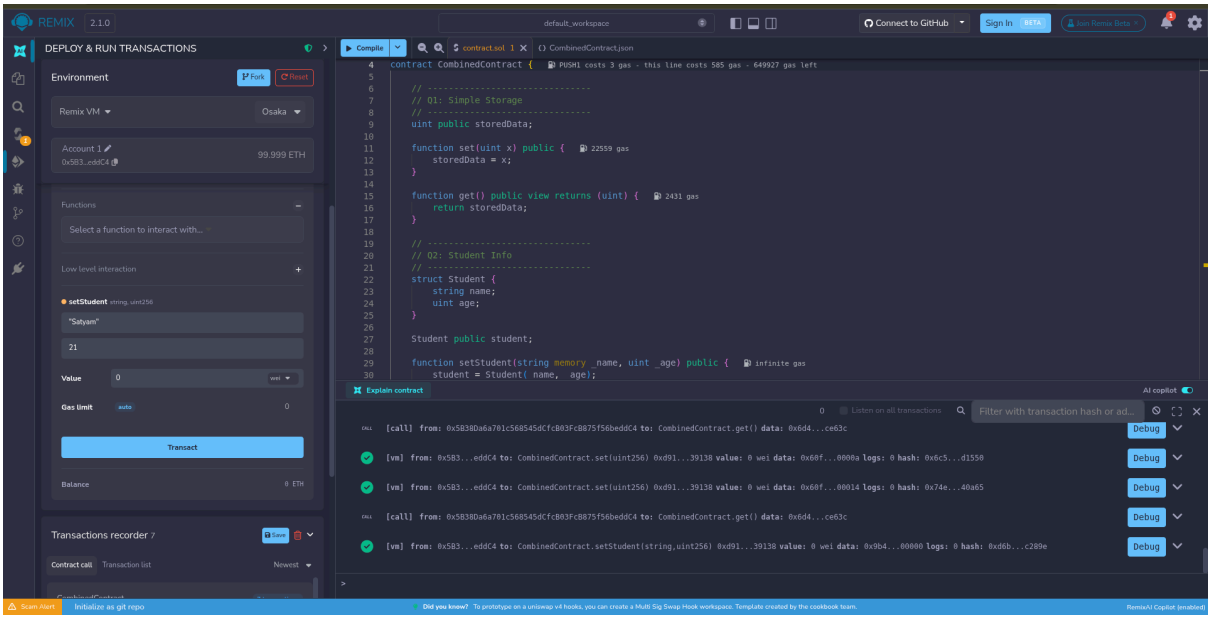
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...0000a logs: 0 hash: 0x87f...dbd77 | Debug

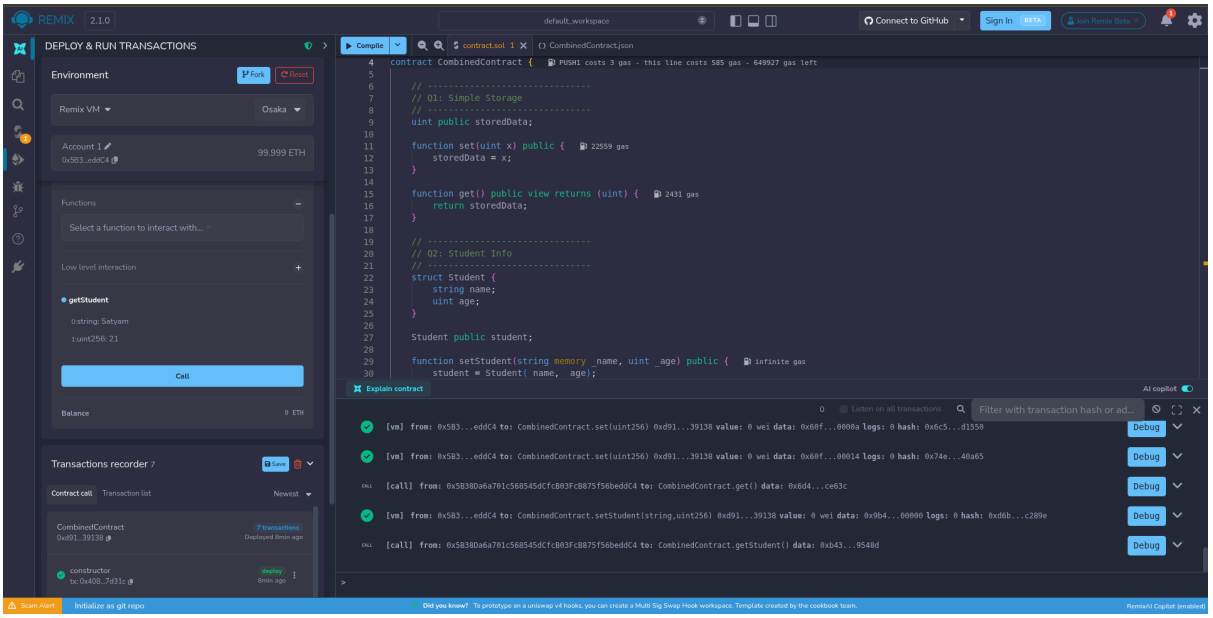
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c | Debug

[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...0000a logs: 0 hash: 0x6c5...d1550 | Debug

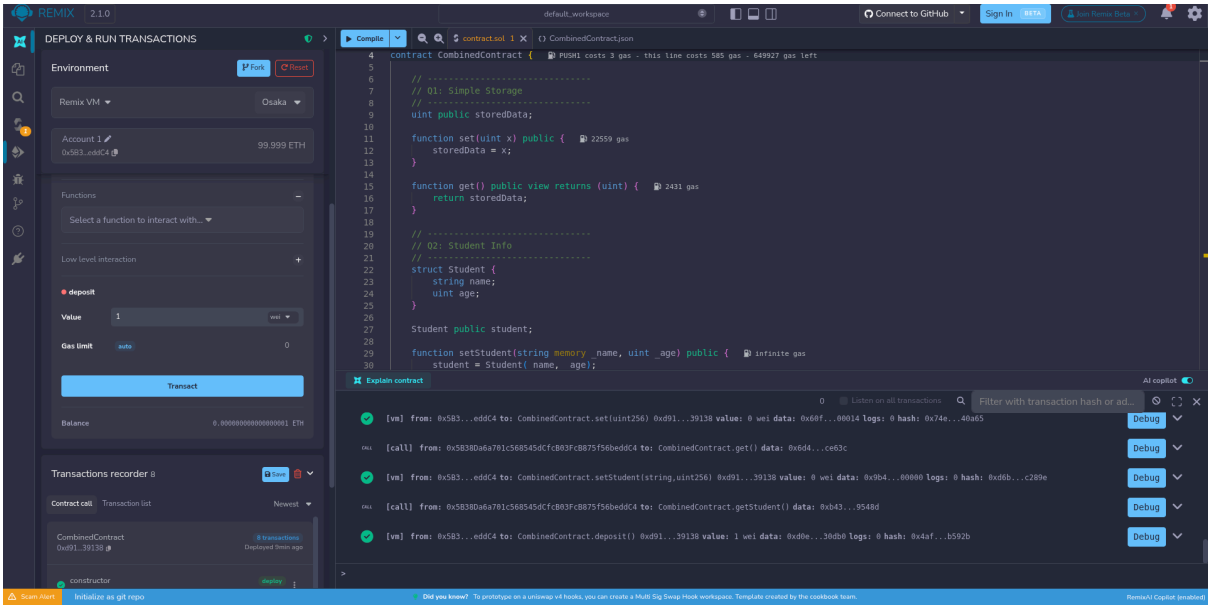
Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

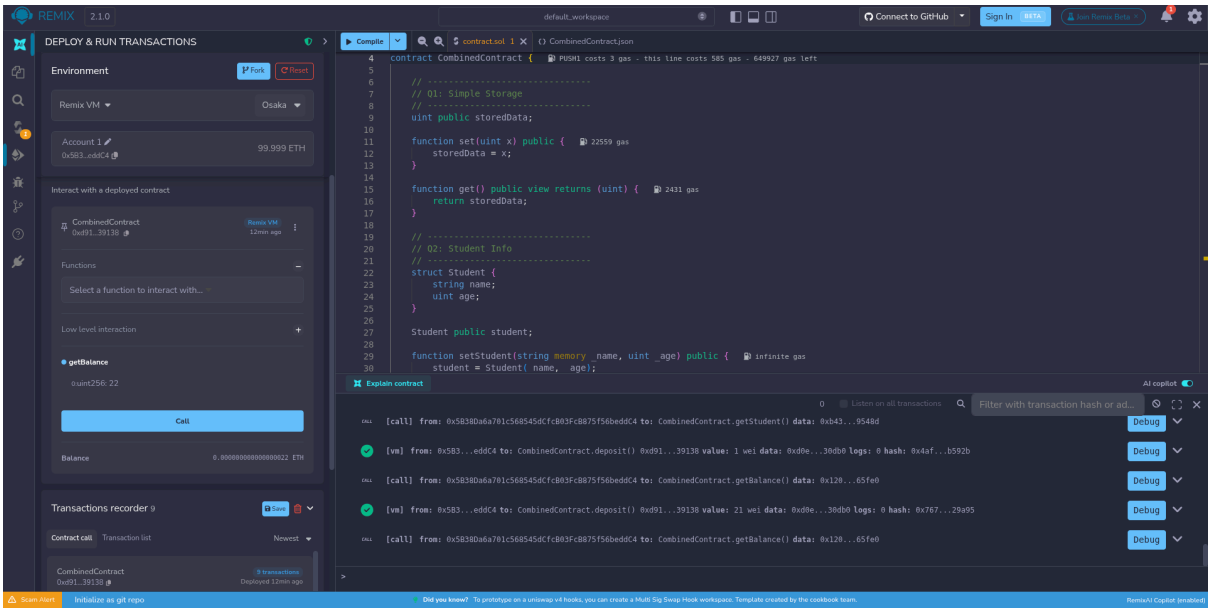
Q2


REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Functions
Select a function to interact with...
Low level interaction
setStudent string, uint256
"Satyam"
21
Value
0
wei
Gas limit
auto
0
Transact
Balance
0 ETH
Transactions recorder 7
Contract call
Transaction list
Newest
contract.sol 1
CombinedContract.json
contract CombinedContract {
PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left
// Q1: Simple Storage
uint public storedData;
function set(uint x) public {
22559 gas
storedData = x;
function get() public view returns (uint) {
2431 gas
return storedData;
// Q2: Student Info
struct Student {
string name;
uint age;
Student public student;
function setStudent(string memory _name, uint _age) public {
infinite gas
student = Student(name, age);
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...0000a logs: 0 hash: 0x6c5...d1550
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...00014 logs: 0 hash: 0x74e...40a65
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: CombinedContract.setStudent(string,uint256) 0xd91...39138 value: 0 wei data: 0x9b4...00000 logs: 0 hash: 0xd6b...c289e
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)


REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Functions
Select a function to interact with...
Low level interaction
getStudent
0:string: Satyam
1:uint256: 21
Call
Balance
0 ETH
Transactions recorder 7
Contract call
Transaction list
Newest
CombinedContract
0xd91...39138
7 transactions
Deployed 8min ago
constructor
tx: 0x408...7d31c
deploy
8min ago
contract.sol 1
CombinedContract.json
contract CombinedContract {
PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left
// Q1: Simple Storage
uint public storedData;
function set(uint x) public {
22559 gas
storedData = x;
function get() public view returns (uint) {
2431 gas
return storedData;
// Q2: Student Info
struct Student {
string name;
uint age;
Student public student;
function setStudent(string memory _name, uint _age) public {
infinite gas
student = Student(name, age);
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...0000a logs: 0 hash: 0x6c5...d1550
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...00014 logs: 0 hash: 0x74e...40a65
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: CombinedContract.setStudent(string,uint256) 0xd91...39138 value: 0 wei data: 0x9b4...00000 logs: 0 hash: 0xd6b...c289e
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.getStudent() data: 0xb43...9548d
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)

Q3


REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Functions
Select a function to interact with...
Low level interaction
deposit
Value
1
wei
Gas limit
auto
0
Transact
Balance
0.000000000000000001 ETH
Transactions recorder 8
Save
Contract call
Transaction list
Newest
CombinedContract
0xd91...39138
8 transactions
Deployed 9min ago
constructor
deploy
Compile
contract.sol 1
CombinedContract.json
contract CombinedContract {
PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left
// ----------
// Q1: Simple Storage
// ----------
uint public storedData;
function set(uint x) public {
22559 gas
storedData = x;
function get() public view returns (uint) {
2431 gas
return storedData;
// ----------
// Q2: Student Info
// ----------
struct Student {
string name;
uint age;
Student public student;
function setStudent(string memory _name, uint _age) public {
infinite gas
student = Student(_name, _age);
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[vm] from: 0x5B3...eddC4 to: CombinedContract.set(uint256) 0xd91...39138 value: 0 wei data: 0x60f...00014 logs: 0 hash: 0x74e...40a65
Debug
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: CombinedContract.setStudent(string,uint256) 0xd91...39138 value: 0 wei data: 0x9b4...00000 logs: 0 hash: 0xd6b...c289e
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.getStudent() data: 0xb43...9548d
[vm] from: 0x5B3...eddC4 to: CombinedContract.deposit() 0xd91...39138 value: 1 wei data: 0xd0e...30db0 logs: 0 hash: 0x4af...b592b
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)


REMIX 2.1.0
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Interact with a deployed contract
CombinedContract
0xd91...39138
Remix VM
12min ago
Functions
Select a function to interact with...
Low level interaction
getBalance
0:uint256: 22
Call
Balance
0.000000000000000022 ETH
Transactions recorder 9
Save
Contract call
Transaction list
Newest
CombinedContract
0xd91...39138
9 transactions
Deployed 12min ago
Compile
contract.sol 1
CombinedContract.json
contract CombinedContract {
PUSH1 costs 3 gas - this line costs 585 gas - 649927 gas left
// ----------
// Q1: Simple Storage
// ----------
uint public storedData;
function set(uint x) public {
22559 gas
storedData = x;
function get() public view returns (uint) {
2431 gas
return storedData;
// ----------
// Q2: Student Info
// ----------
struct Student {
string name;
uint age;
Student public student;
function setStudent(string memory _name, uint _age) public {
infinite gas
student = Student(_name, _age);
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.getStudent() data: 0xb43...9548d
[vm] from: 0x5B3...eddC4 to: CombinedContract.deposit() 0xd91...39138 value: 1 wei data: 0xd0e...30db0 logs: 0 hash: 0x4af...b592b
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.getBalance() data: 0x120...65fe0
[vm] from: 0x5B3...eddC4 to: CombinedContract.deposit() 0xd91...39138 value: 21 wei data: 0xd0e...30db0 logs: 0 hash: 0x767...29a95
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: CombinedContract.getBalance() data: 0x120...65fe0
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)

LAB Exercise 9

Q1. Create and start a Hyperledger Fabric test network. Display the running containers.
Q2. Create a simple permissioned blockchain using Hyperledger Fabric.
Q3. Create a permissioned blockchain network with two organizations and demonstrate communication between them.

```
Q1

1. Clone Fabric samples repository:
 git clone https://github.com/hyperledger/fabric-samples.git
2. Navigate to test-network:
 cd fabric-samples/test-network
3. Start the network:
./network.sh up
4. Create a channel:
./network.sh createChannel
5. Deploy chaincode:
./network.sh deployCC -ccn basic -ccp
../asset-transfer-basic/chaincode-go -ccl go
6. Display running containers:
 docker ps
```

```
➜ test-network git:(main) ✗ docker ps
CONTAINER ID     IMAGE                                STATUS            PORTS
NAMES
2e7dcb2f3c1d     hyperledger/fabric-peer:2.5.15       Up 4 seconds      7051/tcp
peer0.org1.example.com
b5576e6c0b6a     hyperledger/fabric-peer:2.5.15       Up 4 seconds      9051/tcp
peer0.org2.example.com
c2b3f8a0d3a1     hyperledger/fabric-orderer:2.5.15    Up 5 seconds      7050/tcp
orderer.example.com
8fcbf9b7a8d7     hyperledger/fabric-ca:1.5.17         Up 5 seconds      7054/tcp
ca_org1
7a6d5e4f3b2c     hyperledger/fabric-ca:1.5.17         Up 5 seconds      8054/tcp
ca_org2
```

```
➜ test-network git:(main) ✗ ./network.sh up
Using docker and docker-compose
Starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and
using database 'leveldb' with crypto from 'cryptogen'
Creating network "test-network_default" with the default driver
Creating volume "test-network_orderer.example.com" with default driver
Creating volume "test-network_peer0.org1.example.com" with default driver
Creating volume "test-network_peer0.org2.example.com" with default driver
Creating ca_org1                  ... done
Creating ca_org2                  ... done
Creating orderer.example.com      ... done
Creating peer0.org1.example.com   ... done
Creating peer0.org2.example.com   ... done
Network up and running
```

Q2:

```go
package main

import (
	"fmt"
	"github.com/hyperledger/fabric-contract-api-go/contractapi"
)

type SmartContract struct {
	contractapi.Contract
}

// Create Asset
func (s *SmartContract) CreateAsset(ctx
contractapi.TransactionContextInterface, id string, value string) error
{
	return ctx.GetStub().PutState(id, []byte(value))
}
```

```go
// Read Asset
func (s *SmartContract) ReadAsset(ctx
contractapi.TransactionContextInterface, id string) (string, error) {
	data, err := ctx.GetStub().GetState(id)
	if err != nil {
		return "", err
	}
	if data == nil {
		return "", fmt.Errorf("asset not found")
	}
	return string(data), nil
}

func main() {
	chaincode, err := contractapi.NewChaincode(new(SmartContract))
	if err != nil {
		fmt.Printf("Error create chaincode: %s", err.Error())
		return
	}

	if err := chaincode.Start(); err != nil {
		fmt.Printf("Error starting chaincode: %s", err.Error())
	}
}
```

```
→ test-network git:(main) ✗ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile "$PWD/organizations/ordererOrganizations/example.com/orderers/
orderer.example.cos/org2/tlscacerts/tlsca.example.com-cert.pem" -C mychannel -n basic --peerAddresses
localhost:7051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/tlsca/
tlsca.org1.example.com-cert.pem" -c '{"function":"CreateAsset","Args":["asset1","blue","10","Satyam","500"]}'
```

```
→ test-network git:(main) ✗ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
{
  "ID": "asset1",
  "Color": "blue",
  "Size": "10",
  "Owner": "Satyam",
  "AppraisedValue": "500"
}
```

Q3
steps (below):

1. Use test-network (already includes Org1 and Org2)
2. Install chaincode on both org peers
3. Approve chaincode definition for both orgs
4. Commit chaincode
5. Invoke transaction from Org1: peer chaincode invoke …
6. Query from Org2: peer chaincode query ...

Result: Org1 writes data, Org2 successfully reads it → communication established.

```
➜ test-network git:(main) ✗ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}
{
  "ID": "asset1",
  "Color": "blue",
  "Size": "10",
  "Owner": "Satyam",
  "AppraisedValue": "500"
}
```